Kubernetes

Kubernetes: Do Zero ao Avançado

Curso completo com 4 módulos

Pods • Deployments • Services • Helm • HPA • GitOps • Produção

CloudTrilhas — cloudtrilhas.com.br

Por Ricardo Simines Scopim

Kubernetes — Módulo 01

Fundamentos do Kubernetes

Kubernetes (K8s) é uma plataforma open-source de orquestração de containers que automatiza deploy, escalonamento e gerenciamento de aplicações.

1. Arquitetura

```
CONTROL PLANE: API Server | etcd | Scheduler | Controller Manager
WORKER NODES:  kubelet | kube-proxy | Container Runtime | Pods
```

Componentes ARQUITETURA

API Server	Ponto de entrada para todos os comandos (kubectl)
etcd	Banco chave-valor com todo o estado do cluster
Scheduler	Decide em qual Node um Pod será executado
kubelet	Agente em cada Node que garante containers rodando
kube-proxy	Gerencia regras de rede entre Pods

2. Instalação Local (Minikube)

kubectl CLI

```
# Linux
curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/b
chmod +x kubectl && sudo mv kubectl /usr/local/bin/

# Windows (PowerShell)
curl.exe -LO "https://dl.k8s.io/release/v1.29.0/bin/windows/amd64/kubectl.exe"
```

minikube CLUSTER LOCAL

```
minikube start --driver=docker
minikube status
kubectl cluster-info
kubectl get nodes
```

3. Comandos Essenciais

kubectl get LISTAR

```
kubectl get pods
kubectl get pods -o wide
kubectl get pods -l app=nginx
kubectl get nodes
kubectl get namespaces
```

kubectl run / apply CRIAR

```
kubectl run meu-nginx --image=nginx
kubectl apply -f meu-pod.yaml
kubectl delete -f meu-pod.yaml
kubectl create namespace dev
```

kubectl describe / logs DEBUG

```
kubectl describe pod meu-nginx
kubectl logs meu-nginx
kubectl logs -f meu-nginx
kubectl exec -it meu-nginx -- /bin/bash
```

Pod YAML DECLARATIVO

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: meu-nginx
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.25
    ports:
    - containerPort: 80
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

4. Jobs, CronJobs e DaemonSets

Job BATCH

Executa tarefa até conclusão.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: job-backup
spec:
  completions: 3
  parallelism: 2
  backoffLimit: 4
  template:
    spec:
      containers:
      - name: worker
        image: busybox
        command: ["sh", "-c", "echo done"]
      restartPolicy: Never
```

CronJob AGENDADO

Job em horários agendados (cron).

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup-diario
spec:
  schedule: "0 2 * * *"
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: busybox
            command: ["sh", "-c", "echo backup"]
          restartPolicy: OnFailure
```

Kubernetes — Módulo 02

Deployments, Services e Configuração

Recursos mais usados no dia a dia: Deployments para gerenciar aplicações, Services para rede, ConfigMaps/Secrets para configuração.

1. Deployments

Deployment YAML ESSENCIAL

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
        readinessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 5
        livenessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 15
```

Comandos DEPLOY

```
kubectl apply -f deployment.yaml
kubectl get deploy
kubectl scale deployment app-web --replicas=5
kubectl set image deployment/app-web nginx=nginx:1.26
kubectl rollout status deployment/app-web
kubectl rollout undo deployment/app-web
kubectl rollout history deployment/app-web
```

Probes HEALTH CHECK

```
# livenessProbe → container vivo? (reinicia se falhar)
# readinessProbe → pronto para tráfego?
# startupProbe → app iniciou? (apps lentas)

# Tipos: httpGet, tcpSocket, exec
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
  failureThreshold: 3
```

2. Services

Tipos de Service REDE

ClusterIP	IP interno (padrão) — entre serviços
NodePort	Porta no Node (30000-32767) — dev
LoadBalancer	LB externo — produção em cloud
ExternalName	Mapeia para DNS externo

Service YAML EXEMPLO

```
apiVersion: v1
kind: Service
metadata:
  name: svc-web
spec:
  type: NodePort
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080

# DNS interno:
# svc-web.default.svc.cluster.local
```

3. ConfigMaps e Secrets

ConfigMap CONFIG

```
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=APP_PORT=8080

# Usar no Pod:
envFrom:
- configMapRef:
    name: app-config
```

Secret SENSÍVEL

```
kubectl create secret generic db-creds \
  --from-literal=username=admin \
  --from-literal=password=senha123

# Usar no Pod:
env:
- name: DB_PASS
  valueFrom:
    secretKeyRef:
      name: db-creds
      key: password
```

Kubernetes — Módulo 03

Storage, RBAC, Ingress e Helm

Persistência de dados, controle de acesso, roteamento HTTP e gerenciamento de pacotes com Helm.

1. RBAC e Segurança

RBAC SEGURANÇA

```
# ServiceAccount → Role → RoleBinding
kubectl create sa app-sa
kubectl auth can-i get pods \
  --as=system:serviceaccount:default:app-sa

# Role (namespace) vs ClusterRole (cluster)
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

Network Policies REDE

```
# Deny all por padrão
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]

# Permitir frontend → backend
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - port: 80
```

2. Ingress

Ingress com TLS HTTP ROUTING

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  tls:
  - hosts: [app.local]
    secretName: tls-secret
  rules:
  - host: app.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-service
            port:
              number: 80
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: backend-service
            port:
              number: 80
```

3. Helm — Gerenciador de Pacotes

Comandos Helm PACOTES

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo nginx
helm install meu-nginx bitnami/nginx
helm install app bitnami/nginx -f values.yaml
helm upgrade app bitnami/nginx --set replicas=3
helm rollback app 1
helm list
helm uninstall app
```

HPA AUTOSCALING

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: frontend
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

4. Taints, Tolerations e Affinity

Taints + Tolerations SCHEDULING

```
# Taint no Node (repele pods)
kubectl taint nodes node1 gpu=true:NoSchedule

# Toleration no Pod (permite rodar)
tolerations:
- key: "gpu"
  operator: "Equal"
  value: "true"
  effect: "NoSchedule"

# Efeitos: NoSchedule, PreferNoSchedule, NoExecute
```

Node Affinity SCHEDULING

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: disktype
          operator: In
          values: [ssd]
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchLabels:
          app: web
      topologyKey: kubernetes.io/hostname
```

Kubernetes — Módulo 04

Observabilidade, GitOps e Troubleshooting

Monitoramento com Prometheus, GitOps com ArgoCD, backup do etcd, segurança e resolução de problemas em produção.

1. Observabilidade

3 Pilares: Métricas (Prometheus + Grafana) | Logs (EFK Stack) | Traces (Jaeger / OpenTelemetry)

Prometheus + Alertas MONITORAMENTO

```
# ServiceMonitor — Prometheus descobre serviços
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: app-monitor
spec:
  selector:
    matchLabels:
      app: myapp
  endpoints:
  - port: metrics
    interval: 15s

# PrometheusRule — Alertas
- alert: HighErrorRate
  expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.1
  for: 5m
  labels:
    severity: critical
```

2. Troubleshooting

CrashLoopBackOff

1. `kubectl logs pod --previous`
2. `kubectl describe pod` → Events
3. ExitCode 137 = OOM killed
4. ExitCode 1 = erro da aplicação

⏳ **Pod em Pending**

1. `kubectl describe pod`
2. Recursos insuficientes (CPU/RAM)
3. PVC não encontrado
4. Taint sem toleration

🌐 **Service não conecta**

1. `kubectl get endpoints svc`
2. Verificar labels/selector
3. `kubectl exec pod -- nslookup svc`
4. `kubectl exec pod -- curl svc:port`

3. GitOps com ArgoCD

ArgoCD Application GITOPS

```
# Git como fonte de verdade → ArgoCD detecta mudança → aplica no cluster
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: minha-app
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/user/repo.git
    targetRevision: main
    path: kubernetes/
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

4. Boas Práticas de Produção

Segurança PRODUÇÃO

```
# Security Context
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
  containers:
  - securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: [ALL]

# Pod Security Standards
metadata:
  labels:
    pod-security.kubernetes.io/enforce: restricted
```

etcd Backup DISASTER RECOVERY

```
# Backup
ETCDCTL_API=3 etcdctl snapshot save /tmp/backup.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Verificar
ETCDCTL_API=3 etcdctl snapshot status /tmp/backup.db

# Restore
ETCDCTL_API=3 etcdctl snapshot restore /tmp/backup.db \
  --data-dir=/var/lib/etcd-restored
```

5. Comandos Mais Usados

Categoria	Comandos
Pods	`get, describe, logs, exec, delete, top`
Deploy	`apply, scale, set image, rollout, undo`
Services	`expose, get svc, get endpoints`
Config	`create configmap, create secret`
Debug	`describe, logs --previous, get events`
Cluster	`get nodes, top nodes, drain, cordon`